

Shipping on the Air

Assignment #03 - Software Architecture and Platforms

A.Y. 2025/2026

Fabio Vincenzi

18 agosto 2026

Indice

1	Introduzione	2
2	Event-driven Architecture	2
2.1	Scelta del candidato	2
2.2	Da orchestrazione a coreografia	2
2.3	Lo strato dei canali	3
2.4	Request/reply su un bus di eventi	5
2.5	Il confine dell'architettura a eventi	6
2.6	Eventi di dominio ed Event Sourcing	6
3	Scaling e Service Levels	7
3.1	SLO e SLI	7
3.2	Strumentazione	7
3.3	Verifica degli indicatori	8
3.4	Deployment su Kubernetes	9
3.5	Replicazione	9
3.6	Self-healing osservato	10
4	Impatto sui test	11
5	Limiti noti	11
6	Conclusioni	13

1 Introduzione

Questo documento descrive come il sistema *Shipping on the Air*, raffinato con i micro-services patterns nell'Assignment 2, sia stato ulteriormente evoluto lungo due direzioni: l'adozione di un'architettura **event-driven** per uno dei microservizi, con Apache Kafka come middleware, e la definizione di **service level** misurabili accompagnata da un deployment su **Kubernetes**.

Il dominio non cambia. Restano **order-service**, **delivery-service**, **drone-service** e l'**api-gateway**. Cambia il modo in cui i servizi si coordinano, il modo in cui il sistema viene misurato e la piattaforma su cui viene eseguito.

Il terzo punto della consegna, di natura opzionale, riguardante la riprogettazione ad agenti del software autonomo che controlla il drone, non è stato affrontato: il lavoro si concentra sui due punti obbligatori.

Come nell'Assignment 2, ogni scelta è presentata indicando il problema che risolve, la soluzione adottata e le conseguenze, incluse quelle negative, e le affermazioni sono accompagnate, dove possibile, da misure raccolte sul sistema in esecuzione.

2 Event-driven Architecture

2.1 Scelta del candidato

La consegna chiede di riprogettare *un* microservizio, quello ritenuto il miglior candidato. La scelta è caduta sul **delivery-service**, per tre ragioni di peso crescente.

La prima è di posizione: il delivery-service è il nodo centrale del sistema. Riceve una richiesta dall'order-service quando un ordine viene confermato, si rivolge al drone-service per ottenere un mezzo, e informa entrambi quando la consegna termina. È quindi il punto in cui l'accoppiamento sincrono produce il danno maggiore, e dove eliminarlo produce il beneficio maggiore.

La seconda è di preparazione. Nell'Assignment 2 il servizio era già stato riscritto con **Event Sourcing**: gli eventi di dominio esistono già come concetti di prima classe, l'aggregate è già diviso fra una fase **process** che decide e una **apply** che muta lo stato, e l'event store è già nascosto dietro una porta. Adottare un'architettura a eventi non richiede quindi di inventare gli eventi, ma soltanto di farli uscire dal processo.

La terza è di coerenza concettuale. Event Sourcing ed Event-driven Architecture condividono l'unità di base, il fatto avvenuto, ma la applicano a problemi diversi: il primo alla persistenza, il secondo alla comunicazione. Applicarli allo stesso servizio rende esplicito questo rapporto, che la sezione 2.6 riprende.

2.2 Da orchestrazione a coreografia

Nell'Assignment 2 la creazione di una consegna era **orchestrata**: un componente conosceva l'intera sequenza e la eseguiva chiamando gli altri.

Listing 1: Assignment 2: il servizio chiama la flotta e attende

```
var droneId = droneService.requestAvailableDrone(pickupLat ,
    pickupLng , weightKg);
if (droneId.isPresent()) {
    delivery.assignDrone(droneId.get());
    repository.save(delivery);
}
```

```
}

```

Questo codice non esiste più. Al suo posto il servizio registra il fatto e prosegue; chi possiede la flotta se ne accorge e risponde di propria iniziativa. Il coordinamento non è più scritto in un punto, ma emerge dalle reazioni: è una **coreografia**.

La conseguenza si legge in una sola riga, il costruttore dell'application service:

Listing 2: La firma che riassume l'intero punto 1

```
// Assignment 2
public DeliveryServiceImpl(DeliveryRepository repository,
                           DroneServicePort droneService,
                           OrderServicePort orderService)

// Assignment 3
public DeliveryServiceImpl(DeliveryRepository repository)

```

Il delivery-service non conosce più l'esistenza degli altri servizi. Prima li dichiarava come dipendenze; ora sa soltanto produrre fatti. Al loro posto è comparso il metodo `assignDrone(DeliveryId, String)`, che è la porta d'ingresso per la risposta della flotta.

La sequenza completa è riportata nella tabella 1.

Tabella 1: La coreografia della creazione di una consegna

#	Attore	Azione
1	utente	POST /api/v1/orders/{id}/confirm sull'api-gateway
2	order-service	conferma l'ordine e pubblica su <code>create-delivery-requests</code> , senza iscriversi alle risposte
3	delivery-service	crea la consegna in stato SCHEDULED , pubblica l'esito e annuncia new-delivery-created
4	drone-service	reagisce all'annuncio e pubblica drone-assigned oppure drone-unavailable
5	delivery-service	riceve l'assegnazione e la consegna passa a DRONE_ASSIGNED
6	delivery-service	a consegna ultimata pubblica delivery-completed
7	order e drone	ciascuno reagisce: l'ordine si chiude, il drone torna disponibile

Il passo 2 è deliberato: confermare un ordine non attende che la consegna esista. Ne consegue che il tracking mostra per un breve intervallo lo stato **SCHEDULED** con `droneId` nullo, e solo successivamente **DRONE_ASSIGNED**. Sul deployment Kubernetes la transizione è stata osservata in **2 secondi**.

Questo punto merita attenzione, perché rappresenta un cambiamento di significato e non solo di implementazione. Nell'Assignment 2 lo stato "consegna programmata ma senza drone" era descritto dal QAS di availability come il **comportamento degradato** atteso in caso di guasto del drone-service. Nell'Assignment 3 lo stesso stato è il **comportamento normale**, attraversato da ogni consegna. La consistenza da immediata è diventata *eventual*, ed è il prezzo strutturale dell'architettura a eventi.

2.3 Lo strato dei canali

L'astrazione adottata è quella del laboratorio: un *event channel* corrisponde a un topic Kafka omonimo. Nel modulo `common` sono state introdotte tre classi.

Tabella 2: Le classi del modulo common

Classe	Responsabilità
<code>OutputEventChannel</code>	pubblica un <code>JsonObject</code> , incapsulando un <code>KafkaProducer</code>
<code>InputEventChannel</code>	sottoscrive un canale e invoca un handler per ogni evento
<code>EventChannelsAdmin</code>	cancella i canali, usato per ripulire fra un test e l'altro

Rispetto alla versione minimale vista in laboratorio sono state introdotte due parametrizzazioni, entrambe necessarie in un sistema con più di un servizio.

La chiave di partizione. Un topic Kafka è diviso in partizioni e l'ordinamento è garantito solo *all'interno* di una partizione; i record privi di chiave vengono distribuiti su tutte. Senza chiave, due eventi relativi alla stessa consegna potrebbero quindi essere consegnati in ordine invertito. Passando l'identificatore dell'entità come chiave si ottiene che tutti i suoi eventi ricadano sulla stessa partizione.

Listing 3: La chiave rende l'ordinamento una proprietà per entità

```
public Future<Void> postEvent(String key, JsonObject event) {
    event.put("timestamp", System.currentTimeMillis());
    var record = KafkaProducerRecord.<String, String>create(name,
        key, event.encode());
}
```

Il consumer group. È il parametro che distingue due comportamenti opposti espressi dallo stesso meccanismo. Kafka non elimina un messaggio quando viene letto: mantiene un *offset*, cioè un segnalibro, per ogni combinazione di gruppo, topic e partizione, e all'interno di un gruppo ciascuna partizione ha un solo proprietario. Consumatori in gruppi **diversi** ricevono quindi ognuno l'intero flusso, mentre consumatori nello **stesso** gruppo se lo dividono.

Tabella 3: Verifica sperimentale della semantica dei gruppi, tre messaggi su una partizione

Lettore	Ricevuti	Motivo
gruppo A, prima lettura	3	segnalibro a zero, legge tutto
gruppo A, seconda lettura	0	il suo segnalibro è ormai in fondo
gruppo B, prima lettura	3	segnalibro proprio, indipendente da quello di A
gruppo C, consumatore 1	3	è il proprietario della partizione
gruppo C, consumatore 2	0	stessa partizione, ma non gli appartiene

L'ultima riga della tabella 3 chiarisce un punto che si presta a fraintendimenti: i messaggi non ricevuti non erano stati distrutti, tant'è che il gruppo B li ha letti subito dopo. Semplicemente non erano stati *consegnati* a quel consumatore. Nel sistema ogni servizio usa quindi un gruppo distinto, altrimenti order-service e drone-service, entrambi interessati a **delivery-completed**, si sottrarrebbero gli eventi a vicenda.

Ne discende anche il limite al parallelismo: aggiungere consumatori a un gruppo non aumenta la capacità oltre il numero di partizioni, perché i consumatori eccedenti restano inattivi.

Infine, i consumatori sono configurati con `auto.offset.reset: earliest`: un gruppo privo di segnalibro parte dall'inizio del log anziché dal presente. È la proprietà per cui un servizio riavviato recupera da sé gli eventi persi durante l'assenza.

2.4 Request/reply su un bus di eventi

Un canale di eventi non possiede il concetto di risposta, che HTTP fornisce nativamente. Il pattern adottato, quello del laboratorio, è di associare a ogni operazione **tre canali**: uno di richiesta e due di esito, che trasportano l'informazione altrimenti veicolata dal codice di stato.

Listing 4: La terna di canali per la creazione di una consegna

<code>create-delivery-requests</code>	<code>richiesta</code>
<code>create-delivery-requests-approved</code>	<code>esito positivo</code>
<code>create-delivery-requests-rejected</code>	<code>esito negativo, con la motivazione</code>

Il `delivery-service` espone quattro terne, per creare, avviare, completare e interrogare una consegna.

I canali di esito sono però condivisi da tutti i chiamanti, quindi ogni richiesta porta con sé un `requestId` che la risposta ripete. Nell'api-gateway questo consente di ricostruire una chiamata sincrona a partire da uno scambio asincrono: il proxy registra una risposta pendente sotto l'identificatore, pubblica la richiesta e attende.

Listing 5: Correlazione richiesta-risposta nell'api-gateway

```
private final Map<String, CompletableFuture<JsonObject>> pending
    = new ConcurrentHashMap<>();

private void complete(JsonObject reply) {
    var waiting = pending.remove(reply.getString("requestId"));
    if (waiting != null) {
        waiting.complete(reply);
    }
}
```

L'attesa è limitata da un timeout di 5 secondi, oltre il quale viene sollevata `ServiceNotAvailableException`; il blocco è ammissibile perché il controller invoca la porta all'interno di `executeBlocking`, quindi fuori dall'event loop. Va osservato che una *rejection* non è un errore del sistema: è il modo in cui l'informazione "nessuna consegna per questo ordine" viaggia su un canale, e viene ritradata in un `Optional.empty()`.

Questo componente è l'unica parte in cui il codice si spinge oltre l'esempio del laboratorio, che pubblica la richiesta senza sottoscrivere i canali di esito e lascia il completamento come esercizio.

Validazione. Con HTTP un corpo malformato veniva intercettato dal `BodyHandler` e tradotto in un 400. Un evento è JSON privo di schema, quindi un campo mancante emergerebbe come dereferenziazione nulla all'interno della logica applicativa, e il messaggio dell'eccezione finirebbe nel canale di rifiuto. Sono stati quindi introdotti due validatori espliciti che producono un rifiuto leggibile, per esempio `missing or not numeric field: pickupLat`.

2.5 Il confine dell'architettura a eventi

La consegna consente esplicitamente agli altri microservizi di mantenere l'architettura originale, adattando solo quanto serve a interagire con quello riprogettato. Il confine è stato tracciato come segue.

Tabella 4: Cosa cambia per ciascun servizio

Servizio	Adattamento
delivery-service	interamente a eventi. Dell'API REST resta il solo <code>/health</code> , e il controller passa da circa 190 a 45 righe
api-gateway	mantiene l'API REST verso l'esterno; internamente sostituisce il proxy HTTP con uno a canali
order-service	mantiene la propria architettura. Pubblica la richiesta di consegna e si iscrive a <code>delivery-completed</code>
drone-service	mantiene la propria architettura. Reagisce a <code>new-delivery-created</code> e a <code>delivery-completed</code>

La scelta di privare il delivery-service dell'API REST, invece di adottare una soluzione ibrida con comandi su canali e interrogazioni in HTTP, è motivata dalla coerenza: un servizio che continuasse a esporre l'intera API sincrona sarebbe event-driven solo a metà, e manterrebbe due percorsi distinti per raggiungere lo stesso stato. Il mantenimento di `/health` è invece necessario, perché è il punto su cui si appoggiano le probe di Kubernetes.

Un difetto emerso proprio durante la transizione conferma il rischio dei due percorsi: finché entrambi erano attivi, una consegna creata via REST non riceveva alcun drone, perché l'annuncio `new-delivery-created` partiva soltanto dal controller a eventi. Che un fatto di dominio dipenda dalla porta d'ingresso utilizzata è fragile, e la rimozione del percorso REST lo elimina alla radice.

2.6 Eventi di dominio ed Event Sourcing

Oltre ai canali di richiesta, ogni consegna pubblica i propri eventi di dominio su un canale dinamico `delivery-{id}-events`. Il meccanismo rispetta l'architettura esagonale: una porta `DeliveryEventPublisher` nel livello applicativo e un adapter Kafka nell'infrastruttura.

Il punto interessante è il luogo dell'invocazione, all'interno del salvataggio del repository:

Listing 6: Due consumatori derivati dallo stesso stream di eventi

```
newEvents.forEach(event -> {
    projector.project(delivery.getId(), event);
    publisher.publish(delivery.getId(), event);
});
```

Il proiettore aggiorna il read model introdotto con CQRS nell'Assignment 2; il publisher alimenta il canale. Entrambi sono **dati derivati** dalla stessa fonte di verità, cioè lo stream di eventi, ed entrambi sono ricostruibili da essa. È la risposta concreta alla domanda su come Event Sourcing ed Event-driven Architecture si relazionino: il primo produce lo stream, il secondo lo distribuisce.

L'implementazione predefinita della porta è `NO_OP`, il che permette agli scenari in cui gli eventi vanno persistiti ma non annunciati, come i test unitari, di non dipendere da un broker.

3 Scaling e Service Levels

3.1 SLO e SLI

Entrambi gli obiettivi sono definiti sull'**api-gateway**, unico componente esposto all'esterno e quindi il punto più vicino a ciò che l'utente percepisce, secondo la raccomandazione delle Lab Notes di formulare la disponibilità *in terms of the user experience*. Non è definito alcun SLA, perché il sistema non ha controparti contrattuali.

Tabella 5: Gli obiettivi e i rispettivi indicatori

SLO 1	almeno il 99,5% delle richieste servito con successo su 30 giorni
SLI 1	percentuale di richieste con status inferiore a 400 sul totale
SLO 2	95esimo percentile del tempo di risposta inferiore a 1 secondo
SLI 2	percentile calcolato dai bucket dell'istogramma delle durate

Un 4xx è considerato un fallimento, perché dal punto di vista dell'utente la richiesta non ha prodotto quanto atteso. Le chiamate a `/health` sono invece **escluse** dal conteggio: sono interrogazioni della piattaforma, ripetute a intervalli fissi e quasi sempre coronate da successo, e includerle gonfierebbe l'indicatore con traffico che nessun utente ha generato.

La soglia di 1 secondo è ereditata dallo scenario di performance formulato nell'Assignment 1, che il report dell'Assignment 2 aveva dovuto dichiarare **non verificabile**, perché le metriche disponibili erano counter e gauge: indicavano quante richieste fossero state servite, non quanto tempo avessero impiegato. Si usa il percentile e non la media perché, come osservano le Lab Notes, una media bassa può nascondere una minoranza di richieste molto lente, che sono esattamente quelle che gli utenti notano.

Error budget. Un obiettivo del 99,5% dichiara implicitamente che lo 0,5% può fallire. Su una finestra di 30 giorni, pari a 43.200 minuti, la quota ammessa è di **216 minuti**, circa tre ore e mezza. È il meccanismo che rende operativo un obiettivo numerico: finché resta margine si può rilasciare e sperimentare, quando è esaurito conviene stabilizzare.

3.2 Strumentazione

Le due metriche che alimentano gli indicatori sono esposte dall'api-gateway sulla porta 9492.

Tabella 6: Metriche introdotte nell'Assignment 3

Metrica	Tipo	Contenuto
<code>gateway_requests_total</code>	counter con label	richieste servite, separate per esito
<code>gateway_request_duration_seconds</code>	histogram	distribuzione dei tempi di risposta

I bucket dell'istogramma sono 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10 secondi. Il bound esatto a 1 secondo è deliberato: `histogram_quantile` interpola fra i bucket, quindi la stima è tanto più accurata quanto più un bound cade sul valore di interesse, che qui è la soglia dello SLO.

La label è una sola, l'esito. Si era valutato di aggiungere il path della richiesta, ma le rotte contengono identificatori, e usarli come valori di label farebbe crescere illimitatamente il numero di serie temporali, problema noto come esplosione della **cardinalità**.

La misura avviene a risposta scritta, tramite `addBodyEndHandler`, che è l'unico momento in cui lo status code è determinato e la durata è completa.

Listing 7: Le due query PromQL degli indicatori

```
sum(rate(gateway_requests_total{outcome="success"}[5m]))
/ sum(rate(gateway_requests_total[5m]))

histogram_quantile(0.95,
  sum by (le) (rate(gateway_request_duration_seconds_bucket[5m]))
)
```

Anche i bucket di un istogramma sono counter, quindi vanno letti attraverso `rate()` e non nel loro valore assoluto.

3.3 Verifica degli indicatori

Gli indicatori sono stati misurati sul deployment Kubernetes, sottoponendo il gateway a un carico generato dall'interno del cluster e composto, per ogni ciclo, da due richieste valide e una destinata a fallire.

Tabella 7: Valori osservati

Indicatore	Atteso	Misurato
disponibilità	0,667, ossia due successi su tre	0,6698
latenza p95	inferiore a 1 s	0,0141 s

La coincidenza fra valore atteso e valore misurato per il primo indicatore è la verifica che la metrica conteggia ciò che dichiara di conteggiare.

Arrivare a queste misure ha richiesto tre correzioni al metodo, che vale la pena riportare perché non sono difetti dell'implementazione ma proprietà del **modello pull** di Prometheus, e sono il tipo di errore che rende un indicatore silenziosamente falso.

Un carico concentrato non è misurabile. Inviando tutte le richieste in un unico momento fra due scrape, la serie temporale dei fallimenti nasce già al suo valore finale e prosegue costante. La funzione `rate()` scarta il primo campione di una serie appena comparsa, non potendo sapere quanto valesse in precedenza: osserva un incremento nullo e restituisce zero. L'indicatore di disponibilità risultava pari a 1 in presenza di sessanta fallimenti reali. Il carico deve durare più di un intervallo di scrape.

Un indicatore va letto durante il traffico. Interrogando a carico concluso, la finestra temporale è interamente piatta e `rate()` restituisce nuovamente zero. È la differenza fra un counter, che è cumulativo, e un tasso, che è locale alla finestra.

Il port-forward non bilancia. Le prime misure attribuivano tutte le richieste a una sola replica del gateway. La causa è che `kubectl port-forward` verso un Service si aggancia a un singolo pod. Colpendo il ClusterIP dall'interno del cluster il carico si è distribuito sulle tre repliche in proporzione 50, 37 e 39.

3.4 Deployment su Kubernetes

Il sistema è descritto da sette manifest nella cartella `kubernetes/`: un namespace dedicato, i quattro servizi, l'infrastruttura Kafka con Zookeeper, e Prometheus. Il cluster locale utilizzato è **kind**. Poiché le immagini sono costruite localmente e non pubblicate su un registry, vanno caricate nel nodo con `kind load docker-image` e i Deployment specificano `imagePullPolicy: IfNotPresent`.

L'osservazione più rilevante riguarda ciò che **non** è stato necessario fare. Le variabili d'ambiente dei manifest sono identiche a quelle del Docker Compose, perché i servizi si individuano per nome di Service e il DNS interno di Kubernetes li risolve esattamente come faceva quello di Docker. Il pattern **Externalized Configuration**, adottato nell'Assignment 2, ha quindi reso il passaggio di piattaforma possibile **senza modificare una riga di codice Java**.

Rispetto all'esempio del laboratorio, che deploia un sistema con soli Deployment e Service, i manifest si discostano su cinque punti, di cui tre necessari al funzionamento.

Tabella 8: Scostamenti dall'esempio di laboratorio

Aggiunta	Motivazione
variabili d'ambiente	necessarie: senza, i servizi non si individuano
<code>imagePullPolicy</code>	necessaria: le immagini sono locali, non esiste un registry
broker Kafka	necessario: è il middleware richiesto dal punto 1
liveness e readiness probe	discrezionale, motivata dalle Lab Notes e resa possibile da <code>/health</code>
Prometheus	discrezionale: nessun laboratorio lo deploia su Kubernetes

La distinzione fra le due probe merita di essere esplicitata, perché agiscono su piani diversi: la *liveness* accerta che il processo sia vivo e in caso contrario lo sostituisce, mentre la *readiness* accerta che sia pronto a servire e in caso contrario lo rimuove dagli endpoint del Service senza ucciderlo. Un servizio che sta ancora sottoscrivendo i propri canali è vivo ma non pronto.

3.5 Replicazione

Il numero di repliche è portato a **tre solo per l'api-gateway**, che è l'unico componente privo di stato proprio. Gli altri restano a una replica perché il loro stato risiede in memoria: con più repliche il drone-service assegnerebbe lo stesso drone a due consegne diverse. È il limite già dichiarato nel report dell'Assignment 2, che il presente lavoro non rimuove ma circonda.

Va precisato che cosa questa replicazione produce, per non attribuirle più di quanto dia. Produce **disponibilità**: la caduta di un'istanza non interrompe il servizio, come mostrato nella sezione successiva. Non produce **throughput**, per tre ragioni concorrenti.

Il collo di bottiglia resta a valle, perché ogni operazione di dominio converge su un `delivery-service` a replica singola. Il canale di risposta è in `broadcast`, quindi il lavoro di lettura da Kafka viene moltiplicato per il numero di repliche anziché diviso. E ogni richiesta in volo trattiene un thread fino alla risposta o al timeout. Aumentare la capacità del sistema richiederebbe prima di estrarre lo stato del `delivery-service`, poi di aumentare le partizioni dei suoi canali.

Replicare il gateway ha reso osservabili due difetti che con una sola replica non si sarebbero mai manifestati.

Consumer group condiviso. Le tre repliche condividevano il gruppo `api-gateway`, quindi Kafka distribuiva le partizioni fra loro e la risposta a un `requestId` poteva essere consegnata a una replica diversa da quella in attesa, che sarebbe andata in timeout. Il canale di risposta è un **`broadcast`**, non un carico da distribuire: il gruppo è stato reso **per istanza**, così che ogni replica riceva tutte le risposte e scarti quelle non proprie. Gli altri tre servizi mantengono il gruppo fisso, che per essi è corretto.

Listing 8: Un gruppo per istanza, non per servizio

```
static final String CONSUMER_GROUP = "api-gateway-" + UUID.
    randomUUID();
```

Scrape attraverso un ClusterIP. Un Service ClusterIP effettua bilanciamento, quindi Prometheus avrebbe interrogato una replica diversa a ogni scrape, producendo una singola serie temporale i cui counter oscillerebbero fra tre processi distinti. La soluzione adottata è un secondo Service **`headless`**, privo di indirizzo proprio, il cui nome DNS risolve agli indirizzi di tutti i pod, abbinato a `dns_sd_configs` nella configurazione di Prometheus. La verifica sul cluster mostra **sei target attivi, di cui tre distinti per il gateway**, uno per replica.

Era stata valutata l'alternativa `kubernetes_sd_configs` con `role: pod`, più generale, ma richiede ServiceAccount, Role e RoleBinding, ossia RBAC, che eccede il perimetro introduttivo del materiale del corso.

3.6 Self-healing osservato

Il beneficio più immediato della piattaforma è il control loop che riporta lo stato osservato a quello dichiarato. È stato osservato in due occasioni.

La prima provocata: eliminando manualmente un pod dell'`api-gateway` sotto carico, il ReplicaSet ne ha creato immediatamente un altro, divenuto pronto in **45 secondi**, mentre le due repliche superstiti continuavano a servire senza interruzione.

La seconda spontanea, e per questo più significativa. Ricreando il broker, il nuovo pod è partito mentre il nodo effimero della registrazione precedente era ancora presente in Zookeeper, e Kafka è terminato con `NodeExistsException`. Kubernetes lo ha riavviato e al secondo tentativo la registrazione è riuscita. Un avvio con Docker Compose si sarebbe arrestato in quel punto.

Una probe che impediva il funzionamento. La configurazione iniziale prevedeva sul broker una `readinessProbe` di tipo `exec` basata su `kafka-broker-api-versions`. Sul cluster il pod non raggiungeva **mai** lo stato pronto: risultava in esecuzione da 36 minuti nello stato 0/1, con la conseguenza che il Service `broker` non possedeva endpoint e nessun

servizio poteva raggiungerlo. La rimozione della probe risolve il problema. È un esempio di come uno strumento pensato per aumentare l'affidabilità possa, se calibrato male, impedire l'avvio.

A verifica conclusa, i pod applicativi hanno mantenuto lo stato pronto per **18 ore consecutive senza alcun riavvio**.

4 Impatto sui test

Il passaggio a un flusso non sincrono ha effetti diretti sulla verifica, perché i test dell'Assignment 2 asserivano immediatamente dopo l'azione, assumendo che al ritorno della chiamata l'intera catena fosse conclusa. Tale assunzione non vale più per costruzione.

I test sono stati quindi riformulati in modo da **attendere che il sistema si assesti**, interrogando ripetutamente entro un budget di tempo anziché verificare una sola volta. Lo scenario end-to-end esprime ora questa attesa nel proprio testo, con la formulazione *the delivery eventually has a drone assigned*.

È stata inoltre introdotta la classe `BrokerAvailability`, che sospende i test dipendenti dal middleware, tramite `Assumptions.abort()`, quando il broker non è raggiungibile: un test che non può essere eseguito deve dichiararsi saltato e non fallito, altrimenti l'esito perde significato.

Va segnalato che i test non sono un requisito del presente assignment; il loro adattamento è stato comunque svolto per non lasciare in eredità una suite non rappresentativa.

5 Limiti noti

Il delivery-service non è replicabile, e il motivo è duplice. Event store, snapshot e read model restano strutture dati in memoria, quindi due repliche avrebbero due verità diverse. A questo si aggiunge un secondo ostacolo, meno ovvio e specifico dell'architettura a eventi: i sette canali in ingresso del servizio sono sottoscritti sotto un unico consumer group, per cui Kafka distribuisce le loro partizioni **fra** le repliche invece di replicarne il consumo.

La cosa è stata verificata sul cluster portando il servizio a due repliche. L'assegnazione osservata è stata la seguente:

Listing 9: Assegnazione dei canali con due repliche del delivery-service

```
pod A    create-delivery-requests, get-delivery-requests, start-
         delivery-requests
pod B    drone-assigned, complete-delivery-requests, drone-
         position-reports, drone-unavailable
```

Il ciclo di vita di una singola consegna viene così spezzato fra due processi con memorie separate: la consegna nasce in A, ma la risposta della flotta arriva in B, che non la conosce. Il comportamento osservato è stato che nessuna consegna riceve più un drone e resta indefinitamente in stato `SCHEDULED`, con il seguente messaggio nel log della replica B:

Listing 10: Esito osservato

```
WARNING: DroneAssigned discarded - Delivery not found: c5c7fdbe
-565e-4e8a-b211-a6ff4d86508d
```

Va sottolineato che il guasto non è dovuto a Kafka, che si è comportato correttamente, né all'architettura a eventi: la stessa configurazione con lo stato in memoria si sarebbe rotta identicamente nella versione sincrona dell'Assignment 2. Rendere il servizio replicabile richiede, nell'ordine, uno store condiviso, un numero di partizioni maggiore di uno, e solo allora l'aumento delle repliche.

La persistenza su file è stata valutata e rinviata. Il passaggio da event store in memoria a event store su file è l'evoluzione più immediata, ed è stato quantificato: circa 80 righe per un convertitore degli eventi e circa 60 per il nuovo adapter, oltre al montaggio di un volume persistente nel manifest. Il dominio non verrebbe toccato, perché l'event store è già definito dietro una porta.

Due avvertenze emerse dall'analisi meritano di essere riportate, perché non sono evidenti.

La prima è che il serializzatore già presente **non è riutilizzabile**. Quello impiegato per pubblicare gli eventi sul canale è deliberatamente parziale: dell'evento di creazione trasmette ordine, peso e data, ma non la rotta, che a un consumatore non serve. La persistenza richiede invece una traduzione **senza perdite**, perché da essa si deve poter ricostruire l'aggregate identico. Notifica e persistenza hanno esigenze diverse e non condividono lo schema.

La seconda è che su Kubernetes il file, da solo, non produce alcun effetto: il filesystem di un container è effimero, quindi senza un volume persistente lo storico sparisce comunque a ogni riavvio del pod.

Va infine chiarito che questa evoluzione darebbe **durabilità, non replicabilità**. Il controllo di concorrenza ottimistica è oggi garantito dall'atomicità di un'operazione su mappa, valida entro una singola JVM; due processi che scrivano lo stesso file non condividono alcun lock. Per la replicazione servirebbe uno store realmente condiviso, in cui il controllo di versione sia imposto da un vincolo del database.

Kafka non è l'event store. Era stata considerata l'ipotesi di utilizzare Kafka anche come event store del delivery-service, il che avrebbe fatto coincidere i due punti della consegna. È stata scartata per contenere il rischio, ma resta una direzione coerente, tanto più che risolverebbe insieme durabilità e condivisione dello stato fra repliche.

Un topic per consegna non è sostenibile su larga scala. Gli eventi di dominio sono pubblicati su un canale dinamico `delivery-{id}-events`, creato alla prima pubblicazione di ciascuna consegna. La soluzione ricalca l'esempio del laboratorio, dove i canali dinamici sono pochi e a vita breve, ma qui il numero di topic cresce linearmente con il numero di consegne, e ogni topic ha un costo in file e metadati sul broker. La forma abituale per un flusso di questo tipo è un unico topic con l'identificatore come chiave, cioè lo stesso meccanismo già impiegato per garantire l'ordinamento.

La readiness probe non riflette la reale disponibilità del servizio. L'endpoint `/health` risponde UP in modo incondizionato, con l'elenco dei controlli vuoto, ed è eredità dell'Assignment 2, quando ogni operazione passava da HTTP e l'affermazione era veritiera. Dopo il passaggio ai canali non lo è più: il deploy del controller a eventi è asincrono e nessuno ne attende il completamento, quindi un pod può risultare pronto mentre il suo consumer group sta ancora completando il rebalance. Il fenomeno è stato osservato dopo un riavvio del delivery-service, con circa 15 secondi di richieste fallite a fronte di un pod

dichiarato pronto dalla piattaforma. Popolare i controlli con lo stato della sottoscrizione ai canali renderebbe la probe veritiera.

La consegna degli eventi è at-least-once. I consumatori usano il commit automatico dell'offset, quindi un processo che elabori un evento e termini prima del salvataggio del segnalibro lo rielaborerebbe alla ripartenza. Gli handler non sono stati progettati per essere idempotenti, il che è accettabile in un prototipo ma andrebbe affrontato in un sistema reale.

La finestra di 30 giorni non è osservabile. Gli obiettivi sono enunciati su 30 giorni perché è la scala su cui un service level ha senso, ma il sistema viene avviato per la durata delle prove e Prometheus non possiede quella storia. Le verifiche usano una finestra di pochi minuti: cambia l'ampiezza del campione, non la forma dell'indicatore.

L'obiettivo di latenza è ampio rispetto ai valori osservati. Il p95 misurato è di circa 14 ms, quindi quasi settanta volte inferiore alla soglia. La soglia è stata mantenuta perché proviene dal quality attribute dichiarato nell'Assignment 1, ma il margine va riportato: un obiettivo largo dichiarato insieme al proprio margine è informativo, lo stesso obiettivo presentato da solo apparirebbe scelto per non essere mai violato.

L'indicatore di disponibilità ha un punto cieco. Conteggia le richieste *a cui il gateway ha risposto*: se il gateway fosse spento, numeratore e denominatore resterebbero entrambi fermi e il rapporto resterebbe elevato. È la trappola descritta dalle Lab Notes nell'esempio Shakespeare. L'indicatore va quindi letto insieme alla metrica `up`, generata automaticamente da Prometheus per ogni target e già impiegata nel QAS di availability dell'Assignment 2: `up` copre il caso “non risponde affatto”, lo SLI il caso “risponde male”.

I canali non sono documentati formalmente. Le API REST dei servizi sono descritte da specifiche OpenAPI, mentre i canali di eventi sono documentati solo dal codice e da questo report. Una descrizione in AsyncAPI renderebbe il contratto verificabile e sarebbe il corrispettivo naturale di quanto già fatto per il lato sincrono.

Il terzo punto della consegna non è stato affrontato. La riprogettazione ad agenti del software di controllo del drone, di natura opzionale, non rientra nel lavoro svolto.

6 Conclusioni

L'assignment ha modificato il sistema lungo due direzioni indipendenti, che si sono però rivelate collegate.

Sul piano della comunicazione, l'adozione dell'architettura a eventi ha sostituito l'**orchestrazione** con la **coreografia**. Il risultato più sintetico è la firma del costruttore dell'application service, passata da tre dipendenze a una: il delivery-service non conosce più l'esistenza degli altri servizi e si limita a produrre fatti. Il prezzo, dichiarato e non nascosto, è il passaggio a una consistenza *eventual*, che rende normale uno stato intermedio che nell'assignment precedente era considerato degradato.

Sul piano operativo, la definizione di service level ha portato a compimento un percorso iniziato nell'Assignment 2. Là un quality attribute era stato dichiarato non verificabile

per insufficienza degli strumenti di misura; qui l'istogramma delle durate lo rende osservabile. Il principio che ne emerge, già enunciato allora, trova conferma: **lo strumento di misura determina quali attributi di qualità è possibile promettere**. Le tre trappole incontrate nel misurare gli indicatori aggiungono un corollario meno ovvio, cioè che possedere lo strumento non basta, perché un indicatore mal osservato può restituire un valore rassicurante e falso.

Il collegamento fra le due direzioni si trova nel deployment. Kubernetes ha reso possibile replicare un componente, e l'atto stesso di replicarlo ha fatto emergere due difetti latenti nella soluzione a eventi, il consumer group condiviso e lo scrape attraverso un servizio bilanciato, entrambi invisibili con una sola istanza. È l'osservazione più utile dell'intero lavoro: una scelta di deployment non è neutrale rispetto all'architettura applicativa, ma ne verifica le assunzioni.